

Laboratorio de Programacion II

Docente: Ing. Erick Amaya

Grupo 5:

Kendrick Joad Gutierrez Maldonado 22611069

Erick Said Arevalo Reyes 22551007

Marlos Edgardo Medina Banegas 22611292

Actividad: **Juego de memoria Pokemon**

31/10/2026

Introduccion

El presente informe tiene como finalidad describir la estructura y el funcionamiento del proyecto Juego de Memoria Pokémon, desarrollado utilizando el lenguaje de programación Java en el entorno NetBeans. Este proyecto consiste en un juego para dos participantes, en el cual deben encontrar parejas de cartas que contienen diferentes Pokémon, usando su memoria y concentración.

Descripción general del funcionamiento del juego

El Juego de Memoria Pokémon está diseñado para dos jugadores, quienes deben encontrar las parejas de cartas iguales dentro de un tablero de 6 filas por 6 columnas, compuesto por un total de 36 cartas.

Al iniciar una partida, el sistema solicita los nombres de los participantes y prepara el tablero, colocando las cartas en posiciones aleatorias. Cada carta permanece oculta hasta que uno de los jugadores la selecciona.

Durante cada turno, el jugador puede descubrir dos cartas. Si ambas contienen el mismo Pokémon, se considera una pareja encontrada y el participante recibe los puntos correspondientes. Las cartas normales otorgan 1 punto, mientras que las cartas especiales otorgan 2 puntos. Cuando las cartas no coinciden, vuelven a ocultarse después de un breve intervalo y el turno pasa al otro jugador.

Este proceso continúa hasta que se encuentran todas las parejas del tablero. Finalmente, el sistema compara los puntajes de ambos participantes y determina quién obtuvo la mayor cantidad de puntos. En caso de empate, se declara un empate entre los jugadores.

Estructura del proyecto y responsabilidades de las clases

El proyecto está organizado mediante diferentes clases, cada una con una función específica dentro del sistema. Esta organización permite distribuir las responsabilidades y facilitar el desarrollo, mantenimiento y comprensión del código.

1. Carta

Clase abstracta que establece las características generales de las cartas. Almacena el nombre del Pokémon y controla si la carta está descubierta, oculta o emparejada. También define el método abstracto para calcular los puntos.

2. CartaNormal

Clase que hereda de Carta y representa las cartas normales del juego. Su función principal es otorgar un punto cuando se encuentra una pareja.

3. CartaEspecial

Clase que hereda de Carta y representa las cartas especiales. A diferencia de las normales, otorga dos puntos cuando se encuentra una pareja.

4. Jugador

Administra la información de cada participante, incluyendo su nombre, cantidad de parejas encontradas y puntaje acumulado.

5. ControlJuego

Clase abstracta que establece las operaciones principales que debe realizar el sistema, como iniciar una partida, controlar los turnos, verificar parejas y finalizar el juego.

6.JuegoPokemon

Es la clase encargada de controlar la lógica principal de la partida. Crea y mezcla las cartas, administra a los jugadores, verifica las parejas encontradas, asigna los puntos y determina cuándo termina el juego.

7. NombreInvalido

Clase de excepción personalizada que permite controlar los errores relacionados con los nombres introducidos por los jugadores.

8. Juego_Pokemon

Contiene el método principal del programa, desde el cual se inicia la ejecución del proyecto.

9. Interfaz gráfica (GUI)

Es el componente visual del sistema. Su función es permitir que los jugadores interactúen con el tablero, seleccionen cartas y visualicen sus turnos, aciertos y puntajes.

Aplicación de los conceptos de programación orientada a objetos

Durante el desarrollo del Juego de Memoria Pokémon se utilizan diferentes conceptos de programación que permiten organizar el código y facilitar el funcionamiento del sistema.

Herencia

Se aplica mediante las clases CartaNormal y CartaEspecial, que heredan los atributos y métodos de la clase principal Carta. Esto permite reutilizar código y establecer características comunes entre los diferentes tipos de cartas.

Clases abstractas

Se utilizan las clases Carta y ControlJuego. Estas sirven como estructuras generales que establecen atributos, métodos y operaciones que deben implementar sus clases hijas.

Polimorfismo

Se aplica mediante el método ganarPuntos(), definido en la clase abstracta Carta e implementado de manera diferente en CartaNormal y CartaEspecial. Esto permite utilizar el mismo método para obtener diferentes resultados dependiendo del tipo de carta.

Recursividad

Se utiliza en métodos que realizan operaciones repetitivas mediante llamadas a sí mismos. En el proyecto se aplica para mezclar las cartas del tablero y contar las cartas que ya han sido emparejadas.

Clonación de objetos

Se implementa mediante la interfaz Cloneable y el método clone() de la clase Carta. Su finalidad es crear copias de las cartas originales para formar las parejas necesarias del juego.

Arreglos

Se utilizan arreglos para almacenar y organizar las 36 cartas del tablero y los dos jugadores. Esto permite acceder a cada elemento mediante su posición y administrar los datos durante la partida.

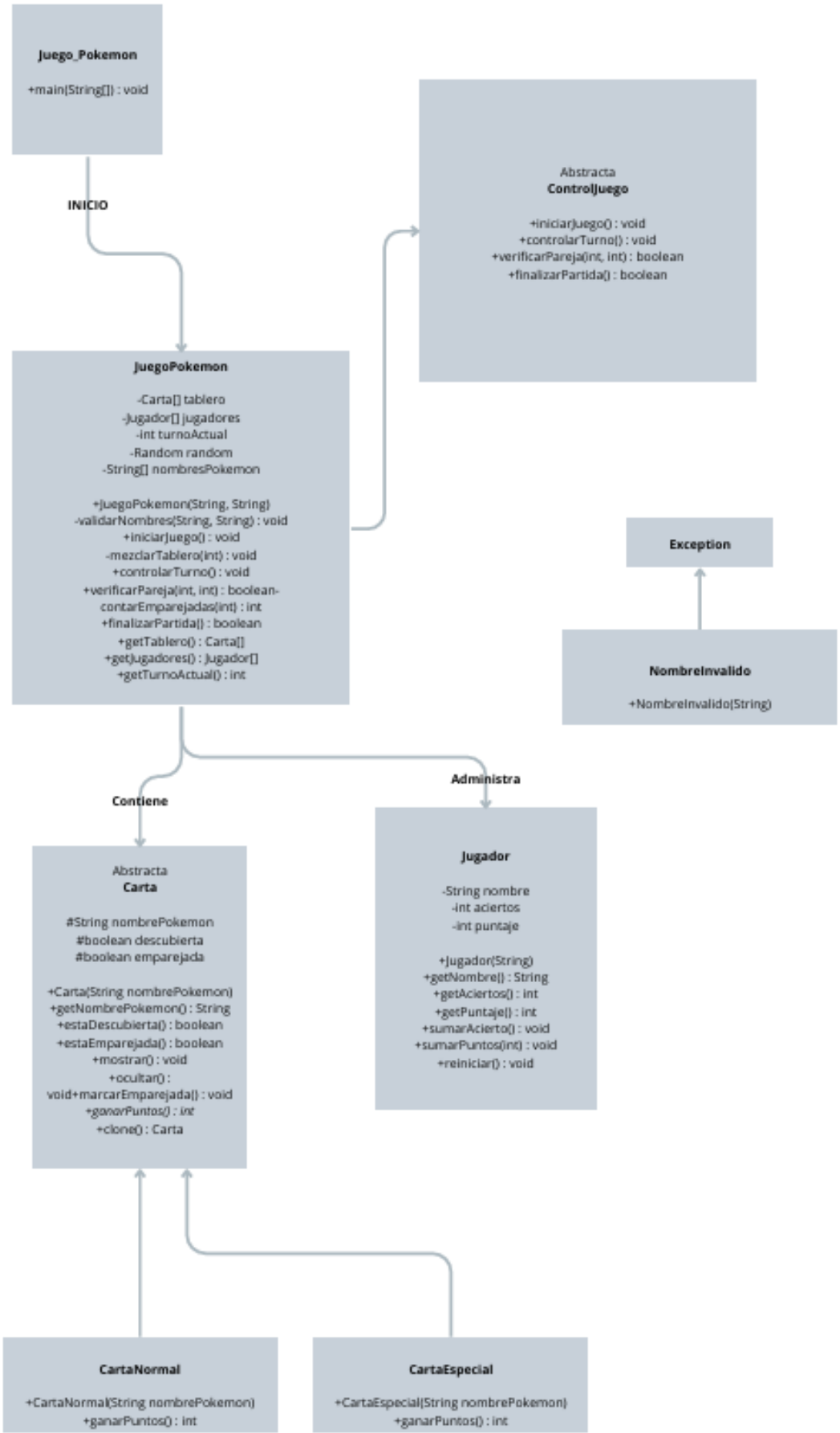
Manejo de excepciones

Se utiliza para controlar situaciones que pueden generar errores durante la ejecución. La clase NombreInvalido permite crear una excepción personalizada para evitar que se registren nombres incorrectos de los jugadores.

Interfaz gráfica

Se utiliza para facilitar la interacción con el usuario mediante elementos visuales. Permite mostrar las cartas, seleccionar parejas y consultar los turnos y puntajes de los participantes.

UML del proyecto



Descripción de los métodos principales

El juego cuenta con diferentes métodos que permiten controlar las operaciones necesarias durante una partida. Cada uno cumple una función específica para mantener el orden y el funcionamiento correcto del juego.

iniciarJuego()

Se encarga de preparar una nueva partida, crear las cartas necesarias, formar las parejas y distribuirlas aleatoriamente en el tablero.

mezclarTablero()

Realiza la mezcla de las cartas mediante un proceso recursivo, cambiando sus posiciones para que las parejas no queden juntas de manera predecible.

controlarTurno()

Administra los turnos de los participantes, permitiendo que cada jugador tenga su oportunidad de descubrir parejas.

verificarPareja()

Comprueba si las dos cartas seleccionadas contienen el mismo Pokémon. Si coinciden, las marca como emparejadas y actualiza los aciertos y puntos del jugador.

ganarPuntos()

Determina la cantidad de puntos que corresponde según el tipo de carta. Las cartas normales otorgan un punto y las especiales dos puntos.

contarEmparejadas()

Utiliza recursividad para contar las cartas que ya fueron emparejadas durante la partida.

finalizarPartida()

Comprueba si todas las cartas del tablero han sido emparejadas para determinar si la partida ha terminado.

clone()

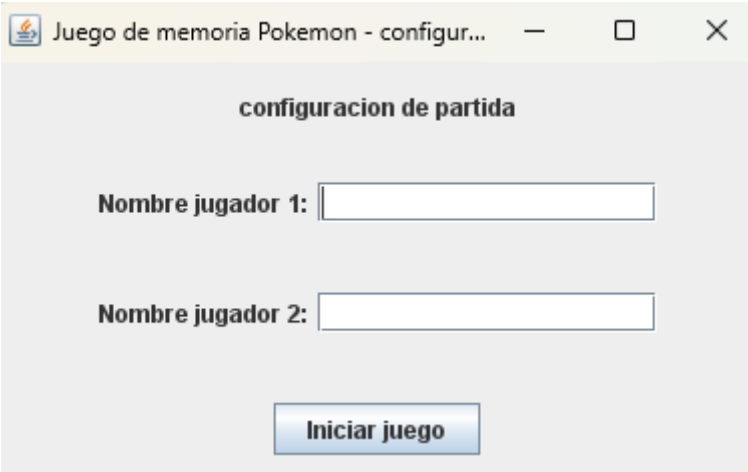
Permite crear copias de los objetos Carta para formar las parejas de Pokémon que se utilizan en el tablero.

Historial de cambios del proyecto



Resultado esperado al ejecutar en Java NetBeans

Pantalla inicial



Juego iniciado

